

SME Form Data Contract

What the review form reads, writes and must never touch. This supersedes the earlier build notes: the validation approach changed, and so did the screen. Read the first two sections before estimating.

What changed since the first brief. A second, independent LLM now answers every case blind. A scheduled job compares the two and accepts agreement automatically. SMEs only see what the automation could not settle, plus two measurement samples. Volume per product fell from about 1,000 cases to about 450.

For the form, three consequences: the screen shows **two model answers side by side**, the SME records **which one was right**, and some cases are shown **blind** with both answers hidden until the SME has committed their own.

Ownership WHO BUILDS WHAT

DATA TEAM	FORM TEAM
Schema, keys, constraints Case sample and reference data Both model runs and their output The triage job that fills the queue Stale-lock reset job Reporting and scorecards	The review screen and its behaviour Claiming and releasing queue rows Writing verdicts and change requests Blind mode Keyboard flow and throughput Supervisor view

Anything needing a schema change comes back to the data team rather than being worked around in the form.

The nine tables the form touches EVERYTHING ELSE IS OUT OF SCOPE

TABLE	ACCESS	WHAT THE FORM USES IT FOR
review_queue	READ / UPDATE	The work list. The only table the form updates.
review_verdict	READ / INSERT	One row per completed review. Never updated.
definition_change	INSERT	Optional, raised from a verdict.
review_batch	READ	Only open batches are served.
case_reference	READ	Case number, title, description, URL, product.
model_run	READ	Finds the paired second-opinion run and the taxonomy version.
model_output	READ	Both answers, with ranked alternates.
auto_assessment	READ	Which fields the two models disagreed on.
skill / v_run_definitions	READ	Skill names for the picker, and the definition wording that <i>this run</i> was given.
sme / sme_entitlement	READ	Which products this reviewer may see.

The join map READ THIS BEFORE WRITING ANY QUERY

The queue row is the anchor

Everything starts from review_queue. It carries case_number and run_id, and queue_id is what the verdict hangs off — not the case number.

Product comes through the batch

review_queue.batch_id → review_batch.product_code.
There is deliberately no product column on the queue.

run_id is always the primary run

Find the second opinion through model_run.paired_run_id. Never assume a fixed naming convention for run IDs.

Definitions are resolved per run

Read v_run_definitions for the queue row's run_id. It returns the exact wording that run was given. Never read the current skill dimension directly — a cycle 1 case must show cycle 1 wording.

Query 1 — claim the next case MUST BE ATOMIC

```
-- Candidate selection
SELECT TOP 1 q.queue_id
FROM validation.review_queue q
JOIN validation.review_batch b
    ON b.batch_id = q.batch_id AND b.run_id = q.run_id
JOIN validation.case_reference c
    ON c.case_number = q.case_number
JOIN validation.sme_entitlement e
    ON e.product_code = c.product_code
    AND e.sme_id = @sme_id
    AND e.active_from <= CAST(GETDATE() AS DATE)
    AND (e.active_to IS NULL OR e.active_to >= CAST(GETDATE() AS DATE))
WHERE q.status = 'Queued'
    AND b.status = 'Open'
    AND NOT EXISTS (
        -- never the same case twice for one reviewer
        SELECT 1 FROM validation.review_queue q2
        JOIN validation.review_verdict v2 ON v2.queue_id = q2.queue_id
        WHERE q2.case_number = q.case_number
            AND q2.run_id = q.run_id
            AND v2Reviewer = @sme_id)
ORDER BY e.is_primary DESC, q.priority, q.created_at;

-- Claim it, and check the row count. Zero means someone else got there first:
-- pick again rather than showing an error.
UPDATE validation.review_queue
    SET status = 'In Review', assigned_sme = @sme_id, locked_at = SYSUTCDATETIME()
    WHERE queue_id = @queue_id AND status = 'Queued';
```

The exclusion is "this reviewer hasn't judged this case", not "this case is unjudged". Slot 2 of a dual review must stay available to someone else.

Query 2 — load the screen READ ONLY

```
-- Header: case, product, run, cycle, taxonomy version, triage flags
SELECT c.case_number, c.case_title, c.case_description, c.case_url,
       p.product_label, r.cycle_number, r.taxonomy_version,    -- label only; wording comes from v_run_definitions
       r.paired_run_id AS second_run_id,
       q.is_blind, q.routing_reason,
       a.skill_agree, a.object_agree, a.defect_agree, a.outcome
FROM validation.review_queue q
JOIN validation.review_batch  b ON b.batch_id = q.batch_id AND b.run_id = q.run_id
JOIN validation.product      p ON p.product_code = b.product_code
JOIN validation.case_reference c ON c.case_number = q.case_number
JOIN validation.model_run    r ON r.run_id = q.run_id
LEFT JOIN validation.auto_assessment a
      ON a.case_number = q.case_number AND a.primary_run_id = q.run_id
WHERE q.queue_id = @queue_id;

-- Both answers, ranked. Two result sets or one with a role column.
SELECT m.run_id, mr.run_role, m.skill_rank, m.skill_code, s.skill_name,
       m.object_failed, m.defect, m.explanation, m.confidence
FROM validation.model_output m
JOIN validation.model_run mr ON mr.run_id = m.run_id
JOIN validation.skill      s ON s.skill_code = m.skill_code
WHERE m.case_number = @case_number
      AND m.run_id IN (@run_id, @second_run_id)
ORDER BY mr.run_role DESC, m.skill_rank;
```

Query 3 — submit ONE TRANSACTION

```
BEGIN TRANSACTION;

INSERT INTO validation.review_verdict
(verdict_id, queue_id, reviewer, reviewed_at, review_seconds,
 skill_correct, corrected_skill_code, rank1_correct, rank_of_correct_skill,
 object_correct, corrected_object, defect_correct, corrected_defect,
 sided_with, error_cat, comment)
VALUES (@verdict_id, @queue_id, @sme_id, SYSUTCDATETIME(), @seconds,
        @skill_correct, @corrected_skill, @rank1_correct, @rank_of_correct,
        @object_correct, @corrected_object, @defect_correct, @corrected_defect,
        @sided_with, @error_cat, @comment);

UPDATE validation.review_queue
SET status = 'Completed', locked_at = NULL
WHERE queue_id = @queue_id;

COMMIT;
```

The database enforces the form's validation. Four check constraints will reject a bad row, so client-side validation is a convenience, not the guarantee: a no on skill, object or defect must carry its correction; any no must carry an error_cat.

A filtered unique index allows **one original verdict per queue row**. A double-submit will fail rather than duplicate. Disable the button on submit and treat a unique-violation as "already saved".

The screen THREE COLUMNS

1 • Case

Number, product, title, description, link to the full record. Clamp the description with an expander rather than letting it push the column.

2 • Both answers

- Primary and second opinion side by side
- Fields that disagree are marked, using `auto_assessment`
- Rank 1 promoted; alternates collapse to one line each and expand to show only what differs from rank 1
- Skill shown as `skill_name`, with definition text on hover or click

3 • Feedback

- Was the correct skill assigned?
- Was the correct skill ranked first? (*only when the run returned more than one skill*)
- Was the failing object correct?
- Was the defect correct?
- Which answer was right: primary, second opinion, both, neither
- Error category, then optional comment

Field rules

- Yes / no / can't tell as three-state toggles, not dropdowns
- Corrections appear only on "no" and clear when the answer flips back
- Error category is mandatory on any "no", hidden otherwise
- Progress counter and cycle number in the header

Blind mode NOT OPTIONAL

When `review_queue.is_blind` is set, column 2 stays hidden until the SME has entered their own skill, object and defect. Then it reveals for comparison and the remaining questions appear.

This covers the calibration set, which measures how accurate the second model is, and the audit sample, which measures how often automated agreement was wrong. Both become meaningless if the reviewer can see the answer first — people agree with an answer in front of them more often than they should. Build it as a mode on the same screen, not a second page.

Queue mechanics STATUS AND LOCKING

- Statuses are fixed: Queued, Assigned, In Review, Completed, Escalated, Superseded. Reporting keys off this list — don't add values without the data team.
- Superseded is set by the data team only. It marks a row made obsolete by a rerun. The form never sets it, and should treat such a row as gone if it's open on screen.
- Skip returns the row to Queued and clears assignee and lock. Escalate sets Escalated and keeps the assignee. They are different actions: escalated cases are diagnostically interesting and must not be silently recycled.
- Locks time out after a configurable period (start at four hours) via a data team job. The form should handle a case being taken away mid-session.
- No deletes, anywhere. Every state is a status change or a new row.
- Supervisors (`sme.is_supervisor`) bypass the entitlement filter, and can reassign a row or release a lock early.

Cycles: the same case comes back MOST LIKELY SOURCE OF BUGS

After a rerun, a case that was reviewed in cycle 1 can appear again in cycle 2 as a **new queue row against a new run**. The case number is identical; the queue_id, run_id, batch_id and model output are all different.

Three rules follow: scope every query by the run_id on the queue row, never by case number alone; key everything the form writes on queue_id; and never cache model output against a case number.

Identifiers carry the cycle: batch B-2-STG9000, queue Q2-000148. Nothing on case_reference marks a case as cycle 2 — which cycles it took part in is derived from which runs produced output for it.

Non-functional THROUGHPUT MATTERS

Keyboard first

A full review should be completable without a mouse. At these volumes it's the single biggest throughput lever.

Prefetch

Load the next case while the SME works the current one. Case load should feel instant.

Volume

About 450 cases per product, roughly 2,700 in total per cycle, concentrated into short windows. Expect bursts, not steady load.

Audit

Reviewer identity and timestamp on every verdict. Capture review_seconds — it's how the next cycle gets sized.

Never written by the form RAISE IT RATHER THAN WORKING AROUND IT

case_group, case_reference, product, skill, skill_definition_history, definition_package, taxonomy_version, model_run, model_output, auto_assessment, review_batch, sme, sme_entitlement. If a design needs one of these written from the review screen, something has been modelled wrong — raise it with the data team first.

To confirm before build starts OPEN QUESTIONS

- Whether corrected object and defect are free text or picked from a controlled list.
- Lock timeout value.
- Whether SMEs may amend their own completed verdicts within a cycle. If yes, the amendment is a new row with supersedes_verdict_id set, never an update.
- Supervisor view: read-only, or reassignment too.
- Whether "can't tell" is captured per field or once per case.
- Whether the second opinion's ranked alternates are shown at all, or only its rank 1.